Magic: The Gathering Multiplayer Network Protocol

A required-scope MTGNP v1.0 implementation in Python: one authoritative server, exactly two TCP clients, terminal controls, the supplied fixed card catalog, mulligans, turns, priority and stack handling, combat, reconnection, and game restart. The client is split into transport, state store, controller, and terminal adapter so a future GUI can reuse the first three layers unchanged.

Requirements

- Python 3.12 or newer
- No third-party runtime packages

Run commands from the repository root.

Start the server and two clients

Open three terminals:

```
python -m server.main --host 127.0.0.1 --port 4444
python -m client.main --host 127.0.0.1 --port 4444 --id player_1 --deck decks/red.json
python -m client.main --host 127.0.0.1 --port 4444 --id player_2 --deck decks/blue.json
```

Add `--verbose` to any process to print each complete JSON PDU. The server also accepts `--reconnect-timeout SECONDS`.

Run the bounded real-socket smoke demo with:

```
python -m scripts.demo_game
```

Terminal commands

Command	Purpose
`ready CARD_ID...`	Submit a deck manually
`keep [CARD_ID...]`	Keep the mulligan hand and bottom the required cards
`mulligan`	Reroll the current opening hand
`land CARD_ID`	Play one land during an active-player main phase
`cast CARD TARGET R=1 X=2`	Cast with color counts; use `-` for no target
`activate SOURCE INDEX TARGET... COST_SOURCE...`	Submit an activated-ability request
`pass`	Pass the current priority token
`attack CARD_ID...`	Declare attackers
`block ATTACKER BLOCKER...`	Declare blockers for one attacker
`damage-order ATTACKER BLOCKER...`	Order multiple blockers
`trigger-order TRIGGER_ID...`	Respond to a trigger-order request
`trigger-choice ID yes/no [TARGET]`	Respond to an optional/targeted trigger request
`discard CARD_ID...`	Discard to seven during cleanup
`concede`	Concede the current game
`state`, `help`, `quit`	Inspect the last snapshot, show help, or exit

The server checks legality. A rejected action produces `ERROR` and does not intentionally update the game state. Clients render only server messages and never calculate outcomes.

Protocol and architecture

Every message is compact UTF-8 JSON prefixed by a four-byte unsigned big-endian payload length. Payloads are capped at 65,535 bytes. `common.pdus` defines all 25 supplied PDU names, their direction, required fields, and all 12 specified error codes.

Each accepted socket has a reader thread. Readers only decode frames and place seat-tagged events on a shared queue. One `GameServer` loop consumes that queue and is the only owner of mutable `GameEngine` state. Per-seat locks serialize outgoing frames. A third simultaneous connection receives an error and is closed.

The server sends personalized snapshots: a player sees their own hand, the opponent hand count, library counts, and all public zones. Neither library order nor the opponent hand is sent. After game over, sockets remain available and both players can submit fresh `PLAYER_READY` messages.

The client boundary is intentionally GUI-ready:

- `ClientNetwork`: TCP, framing, receive thread, heartbeat hooks
- `ClientStateStore`: authoritative snapshot replacement and subscriptions
- `ClientController`: presentation-neutral action methods
- `TerminalUI`: command parsing and rendering only

A GUI should subscribe to `ClientStateStore` and call `ClientController`; it should not import server rules.

Implemented rules

- Lobby setup, unique IDs, supplied-instance deck validation (1–50 cards), server shuffle, 20 life, and random first player
- London-style reroll mulligans, including short-deck handling and card-bottom validation
- Required phase cycle from Untap through Cleanup; first player skips the first-turn draw
- One land per turn, declared color-count payment, atomic server selection/tapping of mana sources, untap and temporary-effect cleanup
- Server-granted request tokens, active/non-active priority transfer, two-pass advancement, and LIFO stack resolution
- Legal targets are checked when cast and again at resolution; invalidated spells fizzle
- Attackers, blockers, multiple-blocker damage order, first strike/double-strike engine support, simultaneous damage, lethal state checks, and no trample carry-over
- `LIFE_ZERO`, `DECK_EMPTY`, `CONCEDE`, and `DISCONNECT` game-over reasons
- Mandatory Gray Merchant enter trigger on the stack

The five required card effects are Lightning Bolt, Counterspell, Unsummon, Giant Growth, and Gray Merchant of Asphodel. All supplied lands provide mana; supplied creatures and permanents retain their listed base statistics and core combat keywords. Other instant, sorcery, triggered, or activated text is intentionally unavailable because complete card-effect coverage is bonus scope.

Supplied card data

`cards.json` contains the 58 definitions from `mtgnp_master_card_list.pdf`. `CardCatalog` derives exactly 312 protocol instance IDs in `<base_id>_<three-digit copy>` form. No outside card database or rule source is used.

Test

```
python -m compileall -q common server client tests scripts
python -m unittest discover -s tests -v
```

The suite covers framing, every PDU contract, catalog totals, hidden information, lifecycle and mulligans, priority/stack behavior, five effects, phases/combat, two-seat connections, GUI-facing client boundaries, and a real-loopback two-client restart.

Specification interpretations

- Formal Section 10 field names and normative prose take precedence over conflicting appendix examples.
- The setup transition is `MULLIGAN -> UNTAP`; the first player does not draw on turn one.
- Opening hands draw up to the cards available. A short-deck mulligan bottoms no more cards than the hand contains.
- Reconnection reuses the existing `PLAYER_READY` PDU with the same player ID because the supplied protocol defines no separate reconnect PDU.
- An attacker blocked by multiple creatures assigns damage in submitted order up to remaining lethal toughness; this required baseline does not carry excess damage to the defending player.
- Combat damage remains marked through End of Combat and is cleared during Cleanup.

Work and AI disclosure

Contributor/tool	Work performed
Course team	Supplied MTGNP RFC, fixed card list, requirements, and final review responsibility
Claude	Interpretation and delegation proposal in the supplied `Prompt.pdf`
OpenAI Codex	Repository inspection, approved design, Python implementation, tests, and documentation

AI output was checked against only the three supplied PDFs and executable tests. No external Magic rules or card sources were used.